Reviewer Pack — Galois Group Action and Resolution Proof Width

Entry d94fe886c87d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-31 22:35:34 UTC

Galois Group Action and Resolution Proof Width

Entry ID: `d94fe886c87d` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-31 22:35:34 UTC

1. Conjecture

Field A (mathematical branch): Galois Theory **Field B** (complexity object): Resolution Proof Complexity

Statement:

> For every Boolean CNF formula φ with m clauses over n variables, the size of its Galois group G(φ) is at most 2^(n + c log(m)) for some constant c > 0, and this upper bound holds for all resolutions of φ.

Rationale (proposer's reasoning):

> Galois theory studies symmetries in algebraic structures, which could potentially expose underlying patterns or redundancies in the structure of resolution proofs. A connection between Galois groups and proof size might reveal new insights into proof complexity, possibly leading to improved lower bounds.

Taxonomy category: `Galois_Theory` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `d9388275875df03a`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For each CNF formula φ, if the correlation coefficient between G(φ) and resolution proof width is ≥ 0.8 AND the mean resolution proof width for formulas with Galois group size ≤ 2^(n + c log(m)) is ≤ 3, then support the conjecture.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM₁	LLM₂
ALGEBRIZATION	UNCERTAIN	0.90	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `ADJACENT_OK` against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `Galois Theory AND resolution proof complexity` - `resolution proof width Galois group action CNF formula` - `upper bound Galois group size Boolean CNF resolution`

Top relevant hits considered: - [http://arxiv.org/abs/2504.04416v1] Meta-Mathematics of Computational Complexity Theory - [http://arxiv.org/abs/0912.1795v7] Galois Theory for H-extensions and H-coextensions - [http://arxiv.org/abs/1611.00827v2] Geometric complexity theory and matrix powering - [http://arxiv.org/abs/1611.05956v2] Galois and Cartan Cohomology of Real Groups - [http://arxiv.org/abs/1202.5725v1] Galois actions on complex braid groups - [http://arxiv.org/abs/hep-ph/0610012v1] Tevatron-for-LHC Report of the QCD Working Group - [http://arxiv.org/abs/1407.5802v2] Galois representations and Galois groups over Q - [s2:10.4230/LIPIcs.ITCS.2024.74] Intersection Classes in TFNP and Proof Complexity

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n, m):
        literals = list(range(-n, 0)) + list(range(1, n+1))
        clauses = []
        for _ in range(m):
            clause = [random.choice(literals) for _ in range(random.randint(2, n))]
            clauses.append(clause)
        return clauses

    def gaussian_elimination(A):
        m, n = len(A), len(A[0])
        rank = 0
        for j in range(n):
            i_max = rank
            for i in range(rank, m):
                if abs(A[i][j]) > abs(A[i_max][j]):
                    i_max = i
            if A[i_max][j] == 0:
                continue
            A[rank], A[i_max] = A[i_max], A[rank]
            for i in range(m):
                if i != rank:
                    factor = -A[i][j] / A[rank][j]
```

```
                    for k in range(n):
                        A[i][k] += factor * A[rank][k]
            rank += 1
        return rank

    def resolution_width(clauses):
        clauses = [set(c) for c in clauses]
        open_clauses = set()
        while True:
            new_clause = None
            for clause1 in open_clauses:
                for literal in clause1:
                    if -literal in clause1:
                        continue
                    for clause2 in clauses:
                        if literal in clause2 and -literal not in clause2:
                            new_clause = clause1.union(clause2)
                            break
                    if new_clause is not None:
                        break
                if new_clause is not None:
                    break
            if new_clause is None:
                return len(open_clauses)
            open_clauses.add(new_clause)

    def galois_group_size(T):
        # Placeholder for Galois group size calculation
        # This is a dummy implementation and should be replaced with actual computation
        return 2 ** (len(T) + 1)

    n = random.randint(5, 40)
    m = random.randint(n, 2 * n)
    cnf = generate_cnf(n, m)
    T = [tuple(c) for c in cnf]

    galois_size = galois_group_size(T)
    width = resolution_width(cnf)

    return {
        "metric_name": "resolution_width",
        "metric_value": width,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

if __name__ == "__main__":
    seeds = [int(s) for s in sys.argv[1:]] or [random.getrandbits(32) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)
```

```
    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(result
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_fai
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
_width', 'metric_value': 0, 'instances_tested': 1, 'n_max': 7, 'conjecture_holds': False, 'counterexa
TRIAL: {'metric_name': 'resolution_width', 'metric_value': 0, 'instances_tested': 1, 'n_max': 25, 'co
TRIAL: {'metric_name': 'resolution_width', 'metric_value': 0, 'instances_tested': 1, 'n_max': 23, 'co
TRIAL: {'metric_name': 'resolution_width', 'metric_value': 0, 'instances_tested': 1, 'n_max': 9, 'con
TRIAL: {'metric_name': 'resolution_width', 'metric_value': 0, 'instances_tested': 1, 'n_max': 38, 'co
TRIAL: {'metric_name': 'resolution_width', 'metric_value': 0, 'instances_tested': 1, 'n_max': 16, 'co
TRIAL: {'metric_name': 'resolution_width', 'metric_value': 0, 'instances_tested': 1, 'n_max': 36, 'co
TRIAL: {'metric_name': 'resolution_width', 'metric_value': 0, 'instances_tested': 1, 'n_max': 19, 'co
TRIAL: {'metric_name': 'resolution_width', 'metric_value': 0, 'instances_tested': 1, 'n_max': 17, 'co

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1a26c373.py", line 109, in <module>
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
                         ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1a26c373.py", line 109, in <genexpr>
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
                              ~^^^^^^^^
KeyError: 'seed'
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed before producing data, which prevents us from evaluating the conjecture's validity. | next: Investigate and fix the crash in the test code to proceed with the evaluation of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	22778
2	preregistration	ollama_remote	glm4:latest	0	0	9594
3	novelty	ollama_remote	glm4:latest	0	0	8225
4	novelty	ollama_remote	glm4:latest	0	0	10091
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21788
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	24918
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	23059
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	24123
9	judge	ollama_remote	glm4:latest	0	0	35072

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 179648 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/d94fe886c87d.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/d94fe886c87d.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/d94fe886c87d.tar.gz` (if generated)